

CaseHierarchyController Class

CaseHierarchyController

Generic Apex controller for caseHierarchyViewer LWC component. Retrieves parent case with all child cases, optionally including financial accounts and successors.

Configurable for any Case hierarchy: Parent Case → Child Cases → (Optional) Financial Accounts → (Optional) Successors

Configuration is controlled via LWC design properties allowing admins to:

- Select which child case fields to display
- Toggle financial account visibility
- Toggle successor visibility

Class Diagram

```
graph TD
    CaseHierarchyController["CaseHierarchyController"]:::mainApexClass
    click CaseHierarchyController "/objects/CaseHierarchyController/"
    CaseHierarchyController_Test["CaseHierarchyController_Test"]:::apexTestClass
    click CaseHierarchyController_Test "/apex/CaseHierarchyController_Test/"

    CaseHierarchyController_Test --> CaseHierarchyController

    classDef apexClass fill:#FFF4C2,stroke:#CCAA00,stroke-width:3px,rx:12px,ry:12px,shadow:drop,color:#333;
    classDef apexTestClass fill:#F5F5F5,stroke:#999999,stroke-width:3px,rx:12px,ry:12px,shadow:drop,color:#333;
    classDef mainApexClass fill:#FFB3B3,stroke:#A94442,stroke-width:4px,rx:14px,ry:14px,shadow:drop,color:#333,font-weight:bold;
    linkStyle 0 stroke:#FF8C00,stroke-width:2px;
```

Apex Code

```
/**
 * CaseHierarchyController
 *
 * Generic Apex controller for caseHierarchyViewer LWC component.
 * Retrieves parent case with all child cases, optionally including financial
 accounts and successors.
 *
 * Configurable for any Case hierarchy:
 * Parent Case → Child Cases → (Optional) Financial Accounts → (Optional) Successors
 *
 * Configuration is controlled via LWC design properties allowing admins to:
```

```

* - Select which child case fields to display
* - Toggle financial account visibility
* - Toggle successor visibility
*/
public with sharing class CaseHierarchyController {

    /**
     * Wrapper for case hierarchy data
     */
    public class CaseHierarchyData {
        @AuraEnabled public Case parentCase { get; set; }
        @AuraEnabled public List<ChildCaseData> childCases { get; set; }
    }

    /**
     * Wrapper for child case with related data
     */
    public class ChildCaseData {
        @AuraEnabled public Case caseRecord { get; set; }
        @AuraEnabled public FinServ__FinancialAccount__c financialAccount { get;
set; }
        @AuraEnabled public List<SuccessorData> successors { get; set; }
    }

    /**
     * Wrapper for successor with allocation
     */
    public class SuccessorData {
        @AuraEnabled public String successorName { get; set; }
        @AuraEnabled public String successorEmail { get; set; }
        @AuraEnabled public String successorPhone { get; set; }
        @AuraEnabled public Decimal allocationPercent { get; set; }
        @AuraEnabled public String pathway { get; set; }
        @AuraEnabled public String status { get; set; }
        @AuraEnabled public Id caseId { get; set; }
    }

    /**
     * Get complete case hierarchy for a parent case
     *
     * @param parentCaseId - ID of the parent case (any Case with child cases)
     * @param childFieldNames - Comma-separated list of Case field API names to
display (for UI filtering)
     * @param includeFinancialAccounts - Whether to include Financial Account data
     * @param includeAccountRoles - Whether to include account role data from
Financial Account Roles
     * @param roleFilterString - Filter roles by this string (comma-separated). If
blank, shows all active roles.
     * @param excludeStatusString - Comma-separated list of Case statuses to exclude
from child cases
     * @return CaseHierarchyData with complete hierarchy
     */

```

```

@AuraEnabled(cacheable=true)
public static CaseHierarchyData getCaseHierarchy(
    Id parentCaseId,
    String childFieldNames,
    Boolean includeFinancialAccounts,
    Boolean includeAccountRoles,
    String roleFilterString,
    String excludeStatusString
) {
    try {
        CaseHierarchyData hierarchyData = new CaseHierarchyData();

        // Query parent case
        hierarchyData.parentCase = [
            SELECT Id, CaseNumber, Subject, Status, CreatedDate,
                AccountId, Account.Name,
                FinServ__FinancialAccount__c,
                FinServ__FinancialAccount__r.Name,
                FinServ__FinancialAccount__r.FinServ__Balance__c,
                Type, RecordType.Name
            FROM Case
            WHERE Id = :parentCaseId
            LIMIT 1
        ];

        // Build child case query with optional status exclusion
        String childQuery = 'SELECT Id, CaseNumber, Subject, Status,
        CreatedDate, ' +
            'ContactId, Contact.Name, Contact.Email, Contact.Phone, ' +
            'AccountId, Account.Name, ' +
            'FinServ__FinancialAccount__c, FinServ__FinancialAccount__r.Name, '
        +
            'FinServ__FinancialAccount__r.FinServ__Balance__c, ' +
            'FinServ__FinancialAccount__r.FinServ__FinancialAccountNumber__c, '
        +
            'Pathway_Confirmed__c, Contact_Established__c, SLA_Status__c, ' +
            'Contact_Attempt_Count__c ' +
            'FROM Case WHERE ParentId = :parentCaseId';

        // Add status exclusion filter if provided
        if (String.isNotBlank(excludeStatusString)) {
            List<String> excludedStatuses = excludeStatusString.split(',');
            List<String> escapedStatuses = new List<String>();
            for (String status : excludedStatuses) {
                escapedStatuses.add('\'' +
                String.escapeSingleQuotes(status.trim()) + '\');
            }
            childQuery += ' AND Status NOT IN (' + String.join(escapedStatuses,
            ',') + ')';
        }

        childQuery += ' ORDER BY CreatedDate ASC';
    }
}

```

```

List<Case> childCases = Database.query(childQuery);

// Conditionally query financial accounts and account roles
Map<Id, FinServ__FinancialAccount__c> financialAccountMap = new Map<Id,
FinServ__FinancialAccount__c>();
Map<Id, List<FinServ__FinancialAccountRole__c>> rolesByAccount = new
Map<Id, List<FinServ__FinancialAccountRole__c>>();

if (includeFinancialAccounts || includeAccountRoles) {
    // Get all related financial account IDs
    Set<Id> financialAccountIds = new Set<Id>();
    for (Case childCase : childCases) {
        if (childCase.FinServ__FinancialAccount__c != null) {
financialAccountIds.add(childCase.FinServ__FinancialAccount__c);
        }
    }

    if (includeFinancialAccounts && !financialAccountIds.isEmpty()) {
        // Query financial accounts with full details
        financialAccountMap = new Map<Id, FinServ__FinancialAccount__c>
([
            SELECT Id, Name, FinServ__Balance__c,
                FinServ__FinancialAccountNumber__c,
                FinServ__Status__c, FinServ__ProductName__c
            FROM FinServ__FinancialAccount__c
            WHERE Id IN :financialAccountIds
        ]);
    }

    if (includeAccountRoles && !financialAccountIds.isEmpty()) {
        // Build dynamic query based on role filter
        String roleQuery = 'SELECT Id, FinServ__FinancialAccount__c,
FinServ__RelatedContact__c, ' +
            'FinServ__RelatedContact__r.Name,
FinServ__RelatedContact__r.Email, ' +
            'FinServ__RelatedContact__r.Phone, FinServ__Role__c,
FinServ__Active__c, ' +
            'SuccessorAllocation__c FROM
FinServ__FinancialAccountRole__c ' +
            'WHERE FinServ__FinancialAccount__c IN :financialAccountIds
AND FinServ__Active__c = TRUE';

        // Add role filter if provided
        if (String.isNotBlank(roleFilterString)) {
            // Parse comma-separated roles
            List<String> roleFilters = roleFilterString.split(',');
            List<String> roleConditions = new List<String>();

            for (String roleFilter : roleFilters) {
                String trimmedRole = roleFilter.trim();

```

```

        roleConditions.add('FinServ__Role__c LIKE \'%' +
String.escapeSingleQuotes(trimmedRole) + '%\');
    }

    roleQuery += ' AND (' + String.join(roleConditions, ' OR ' )
+ ')';
}

    roleQuery += ' ORDER BY SuccessorAllocation__c DESC NULLS LAST';

    // Execute dynamic query
    List<FinServ__FinancialAccountRole__c> allRoles =
Database.query(roleQuery);

    // Group roles by financial account
    for (FinServ__FinancialAccountRole__c role : allRoles) {
        if
(!rolesByAccount.containsKey(role.FinServ__FinancialAccount__c)) {
            rolesByAccount.put(role.FinServ__FinancialAccount__c,
new List<FinServ__FinancialAccountRole__c>());
        }

        rolesByAccount.get(role.FinServ__FinancialAccount__c).add(role);
    }
}

// Build child case data with account roles
hierarchyData.childCases = new List<ChildCaseData>();

for (Case childCase : childCases) {
    ChildCaseData childData = new ChildCaseData();
    childData.caseRecord = childCase;

    // Add financial account if exists
    if (childCase.FinServ__FinancialAccount__c != null) {
        childData.financialAccount =
financialAccountMap.get(childCase.FinServ__FinancialAccount__c);

        // Add account roles for this financial account
        childData.successors = new List<SuccessorData>();
        List<FinServ__FinancialAccountRole__c> accountRoles =
            rolesByAccount.get(childCase.FinServ__FinancialAccount__c);

        if (accountRoles != null) {
            for (FinServ__FinancialAccountRole__c role : accountRoles) {
                SuccessorData roleData = new SuccessorData();
                roleData.successorName =
role.FinServ__RelatedContact__r.Name;
                roleData.successorEmail =
role.FinServ__RelatedContact__r.Email;
                roleData.successorPhone =

```

```

role.FinServ__RelatedContact__r.Phone;
                roleData.allocationPercent =
role.SuccessorAllocation__c;
                roleData.pathway = childCase.Pathway_Confirmed__c;
                roleData.status = childCase.Status;
                roleData.caseId = childCase.Id;

                childData.successors.add(roleData);
            }
        }
    }

    hierarchyData.childCases.add(childData);
}

return hierarchyData;

} catch (Exception e) {
    throw new AuraHandledException('Error retrieving case hierarchy: ' +
e.getMessage());
}
}
}

```

Methods

getCaseHierarchy(parentCaseId, childFieldNames, includeFinancialAccounts, includeAccountRoles, roleFilterString, excludeStatusString)

AURAENABLED

Get complete case hierarchy for a parent case

Signature

```

public static CaseHierarchyData getCaseHierarchy(Id parentCaseId, String
childFieldNames, Boolean includeFinancialAccounts, Boolean includeAccountRoles,
String roleFilterString, String excludeStatusString)

```

Parameters

Name	Type	Description
parentCaseId	Id	- ID of the parent case (any Case with child cases)
childFieldNames	String	- Comma-separated list of Case field API names to display (for UI filtering)
includeFinancialAccounts	Boolean	- Whether to include Financial Account data
includeAccountRoles	Boolean	- Whether to include account role data from Financial Account Roles

roleFilterString	String	- Filter roles by this string (comma-separated). If blank, shows all active roles.
excludeStatusString	String	- Comma-separated list of Case statuses to exclude from child cases

Return Type

CaseHierarchyData

CaseHierarchyData with complete hierarchy

Classes

CaseHierarchyData Class

Wrapper for case hierarchy data

Properties

- parentCase
- AURAENABLED

SIGNATURE

```
public parentCase
```

TYPE

[Case](#)

-
- childCases
 - AURAENABLED

SIGNATURE

```
public childCases
```

TYPE

List<ChildCaseData>

ChildCaseData Class

Wrapper for child case with related data

Properties

- caseRecord
- AURAENABLED

SIGNATURE

```
public caseRecord
```

TYPE

[Case](#)

financialAccount

AURAENABLED

SIGNATURE

```
public financialAccount
```

TYPE

FinServ__FinancialAccount__c

successors

AURAENABLED

SIGNATURE

```
public successors
```

TYPE

List<SuccessorData>

SuccessorData Class

Wrapper for successor with allocation

Properties

successorName

AURAENABLED

SIGNATURE

```
public successorName
```

TYPE

String

successorEmail

AURAENABLED

SIGNATURE

```
public successorEmail
```

TYPE

String

successorPhone

AURAENABLED

SIGNATURE

```
public successorPhone
```

TYPE

String

allocationPercent

AURAENABLED

SIGNATURE

```
public allocationPercent
```

TYPE

Decimal

pathway

AURAENABLED

SIGNATURE

```
public pathway
```

TYPE

String

status

AURAENABLED

SIGNATURE

```
public status
```

TYPE

String

caseId

AURAENABLED

SIGNATURE

```
public caseId
```

TYPE

Id